

Lekcja 1

Temat: Wstęp do programowania w Python

Historia języka programowania Python

Python został stworzony przez **Guido van Rossuma** w latach 1989-1991 w Holandii. Nazwa nie pochodzi od węża, lecz od brytyjskiej grupy komediowej **Monty Python**, której Guido był fanem.

Prace nad nim rozpoczęły się pod koniec **1989** roku w **Centrum Matematyki i Informatyki** (CWI) w Amsterdamie, gdzie van Rossum szukał projektu, który mógłby zająć go w okresie świątecznym. **Python miał być następcą języka ABC**, który również rozwijano w CWI, ale van Rossum chciał stworzyć coś bardziej uniwersalnego i przyjaznego dla programistów.

Kluczowe etapy rozwoju:

- **1991: Premiera pierwszej wersji** – 20 lutego 1991 roku ukazała się publiczna wersja 0.9.0. Python był od początku projektem open source, co pozwoliło na szybki rozwój dzięki wkładowi społeczności.
- **Lata 90.: Rozwój w CWI i CNRI** – Do 1995 roku van Rossum rozwijał Pythona w CWI, a następnie przeniósł się do Corporation for National Research Initiatives (CNRI) w USA, gdzie wydał wersje do 1.6 włącznie. W tym okresie język zyskał kompatybilność z licencją GPL.
- **2000: Python 2.0 i BeOpen.com** – Van Rossum i zespół przenieśli się do BeOpen.com, gdzie wydano Pythona 2.0. Wkrótce potem powstała Python Software Foundation (PSF), która do dziś zarządza rozwojem języka.
- **2008: Python 3.0** – Wprowadzono znaczące zmiany, takie jak lepsza obsługa Unicode, co jednak spowodowało niekompatybilność z Pythonem 2.x. Przejście na "trójkę" trwało lata, a wsparcie dla Pythona 2 zakończyło się w 2020 roku.
- **2018: Rezygnacja van Rossuma** – Guido van Rossum, znany jako "Benevolent Dictator for Life" (BDFL), zrezygnował z roli lidera. Od tego czasu rozwój nadzoruje Steering Council wybrany przez społeczność.

Instalacja interpretera

Windows

1. Wejdź na <https://www.python.org/downloads/windows/>
2. Pobierz **Windows installer (64-bit)**
3. Zainstaluj (opcje standardowe)
4. Wykonaj polecenie: **py -3 -version** . Jeśli zainstalowałeś, poprawnie powinieneś otrzymać komunikat: **Python 3.14.2**
5. Kliknij **Win + I**
6. Kliknij **Aplikacje**, wybierz **Zaawansowane ustawienia aplikacji**. Po czym kliknij **Alias wykonywania aplikacji**
7. Zobaczysz listę. **Przewiń w dół i znajdź:**
 - python.exe
 - python3.exe

Przy nich są przełączniki (ON / OFF). Wyłącz dwie opcje python.exe, python3.exe

Uruchomienie skryptu

Wykonaj polecenie, w katalogu skryptu Python:

python nazwa_skryptu.py

Rozszerzony przykład Hello World

```
import sys
print("Hello, World!")

print("Witaj w świecie Pythona!")
print(f"Używasz Pythona w wersji: {sys.version}")

imie = input("Jak masz na imię? ")
print(f"Cześć {imie}! Miło Cię poznać!")
```

Podstawowe typy komentarzy

Komentarz jednowierszowy - zaczyna się od

```
# To jest komentarz jednowierszowy
x = 10 # To też jest komentarz - po kodzie
```

Komentarz wielowierszowy - używamy trzech cudzysłowów ''' lub """

```
"""
To jest komentarz wielowierszowy.
Może zawierać wiele linii tekstu.
Często używany na początku pliku
lub do dokumentowania funkcji.
"""

'''
To też jest poprawny
komentarz wielowierszowy.
Używaj zgodnie z konwencją projektu.
'''

def oblicz_srednia(liczby):
    """
    Funkcja oblicza średnią arytmetyczną z listy liczb.

    Args:
        liczby (list): Lista liczb całkowitych lub zmiennoprzecinkowych

    Returns:
        float: Średnia arytmetyczna
    """
    return sum(liczby) / len(liczby) if liczby else 0
```

Oznaczanie TODO i FIXME

TODO: Dodać obsługę błędów dla pustej listy
FIXME: Naprawić wyciek pamięci przy dużych danych
HACK: Tymczasowe rozwiązanie - wymaga refaktoryzacji
NOTE: Ważna uwaga dotycząca wydajności
OPTIMIZE: Można przyspieszyć przez caching

```
def przetworz_dane(dane):  
    # TODO: Implementować walidację danych wejściowych  
    # FIXME: Dla pustych danych zwracamy None - do poprawy  
    if not dane:  
        return None # FIXME: Powinien być wyjątek  
  
    # ... reszta kodu ...
```

Wcięcia Pythonie:

```
if x > 0:  
    print(x)
```

- **Dwukropek** : mówi: zaraz będzie blok
- **Wcięcie** mówi, co należy do tego bloku

Przykład błędu

```
x = 5  
if x > 0:  
print("x jest dodatnie") # Błąd: brak wcięcia
```

Wynik:

IndentationError: expected an indented block

Poprawnie

```
x = 5  
if x > 0:  
    print("x jest dodatnie") # wcięcie = 4 spacje (standard)
```

1. Typy numeryczne

int - liczby całkowite

```
# Przykłady  
liczba_calkowita = 42  
ujemna = -10  
duza_liczba = 1_000_000 # podkreślniki dla czytelności, równoważne 1000000  
binarna = 0b1010 # 10 w systemie dziesiętnym  
""" od prawej liczymy  
    •  $1 \times 2^3 = 8$   
    •  $0 \times 2^2 = 0$   
    •  $1 \times 2^1 = 2$   
    •  $0 \times 2^0 = 0$ 
```

```
"""
```

```
szesnastkowa = 0xFF # 255
```

```
# Zastosowania: liczniki, ID, indeksy, obliczenia finansowe
```

```
cena_produktu = 299
```

```
ilosc_sztuk = 5
```

```
id_uzytkownika = 12345
```

float - liczby zmiennoprzecinkowe

```
# Przykłady
```

```
pi = 3.14159
```

```
temperatura = -5.5
```

```
duza_liczba = 1.23e6 #  $1.23 \times 10^6 = 1230000.0$ 
```

```
# Zastosowania: obliczenia naukowe, finanse, pomiary
```

```
cena_netto = 19.99
```

```
waga = 2.5
```

```
srednia = 4.75
```

complex - liczby zespolone

```
# Przykłady
```

```
liczba_zespolona = 3 + 4j
```

```
inna = complex(2, -3) # 2 - 3j
```

```
# Zastosowania: inżynieria, fizyka, przetwarzanie sygnałów
```

```
impedancja = 50 + 100j # zamiast i używa się j (zgodnie z konwencją inż. elektryków).
```

2. Typy tekstowe

str - ciągi znaków

```
# Przykłady
```

```
imie = "Anna"
```

```
nazwisko = 'Kowalska'
```

```
wielolinijkowy = """To jest
```

```
wielolinijkowy
```

```
tekst"""
```

```
f_string = f"Witaj {imie}!" # f-string (Python 3.6+)
```

```
# Zastosowania: komunikaty, dane tekstowe, parsowanie
```

```
email = "user@example.com"
```

```
wiadomosc = "Dziękujemy za zakupy!"
```

```
sciezka = "C:/Users/Dokumenty"
```

3. Typy sekwencyjne

list - listy (mutable)

```
# Przykłady
```

```
lista_liczb = [1, 2, 3, 4, 5]
```

```
lista_mieszana = [1, "dwa", 3.0, True]
```

```
lista_2d = [[1, 2], [3, 4]]
```

```
# Zastosowania: kolekcje elementów, wyniki zapytań, tymczasowe przechowywanie
```

```
zakupy = ["mleko", "chleb", "jajka"]
wyniki_testu = [85, 92, 78, 90]
```

tuple - krotki (immutable)

Tuple (krotka) to uporządkowana kolekcja danych, której **nie da się zmienić po utworzeniu**, idealna do bezpiecznych, stałych struktur.

```
# Przykłady
wspolrzedne = (10, 20)
kolory_rgb = (255, 128, 0)
pojedynczy_element = (5,) # UWAGA: przecinek jest konieczny!
```

```
# Zastosowania: stałe zbiory danych, zwracanie wielu wartości z funkcji
wymiary = (1920, 1080) # rozdzielczość
data_urodzenia = (1990, 5, 15)
```

range - zakresy

range to **typ sekwencyjny**, który reprezentuje **zakres liczb**. **Nie tworzy od razu listy liczb**. Generuje je „w locie” (jest wydajny pamięciowo). Najczęściej używany w pętlach **for**

```
# Przykłady
zakres1 = range(5)      # 0, 1, 2, 3, 4
zakres2 = range(1, 6)   # 1, 2, 3, 4, 5
zakres3 = range(0, 10, 2) # 0, 2, 4, 6, 8
```

```
# Zastosowania: iteracje w pętlach, generowanie indeksów
for i in range(3):
    print(f'Powtórzenie {i}')
```

```
indeksy = list(range(len(["a", "b", "c"]))) # [0, 1, 2]
```

4. Typy mapujące

dict - słowniki

Mapowanie = przyporządkowanie wartości do **unikalnego klucza**. W Pythonie typ mapujący to **dict**. Każdy klucz **może wystąpić tylko raz**. Wartości mogą się powtarzać i mogą być **dowolnego typu**

```
# Przykłady
student = {
    "imie": "Jan",
    "nazwisko": "Kowalski",
    "wiek": 21,
    "oceny": [4.5, 5.0, 4.0]
}
```

```
pusty_sloownik = {}
sloownik_z_kluczami = dict(a=1, b=2)
```

```
# Zastosowania: konfiguracje, bazy danych w pamięci, mapowania
konfiguracja = {
    "host": "localhost",
    "port": 8080,
```

```

    "debug": True
}

ksiazka_telefoniczna = {
    "Anna": "123-456-789",
    "Jan": "987-654-321"
}

```

5. Typy zbiorów

set - zbiory (mutable, unikalne elementy)

```

# Przykłady
zbior_liczb = {1, 2, 3, 3, 2} # {1, 2, 3}
zbior_mieszany = {"a", 1, 3.14, True}

```

```

# Zastosowania: usuwanie duplikatów, operacje matematyczne na zbiorach
unikalne_slowa = set(["kot", "pies", "kot", "ptak"]) # {"kot", "pies", "ptak"}

```

frozenset - zamrożone zbiory (immutable)

frozenset to: **niemodyfikowalny (immutable) zbiór**. Działa podobnie do **set**. Przechowuje **unikalne elementy** oraz podobnie **działa jak tuple**. Frozenset **nie można go zmieniać**.

Różnica: set vs frozenset

set – można zmieniać

```

s = {1, 2, 3}
s.add(4)    # OK
s.remove(2) # OK

```

frozenset – NIE można zmieniać

```

fs = frozenset([1, 2, 3])

```

```

fs.add(4)    # AttributeError
fs.remove(2) # AttributeError

```

```

# Przykłady
zamrozony_zbior = frozenset([1, 2, 3, 3, 2]) # frozenset({1, 2, 3})

```

```

# Zastosowania: klucze w słownikach, stałe zbiory
klucze_slownika = {
    frozenset([1, 2]): "wartość",
    frozenset(["a", "b"]): "inna wartość"
}

```

6. Typy logiczne

bool - wartości logiczne

```
# Przykłady
prawda = True
fałsz = False
wynik_porownania = 10 > 5 # True
```

```
# Zastosowania: warunki, flagi, stany
jest_zalogowany = True
ma_dostep = False
```

7. Typy binarne

bytes - bajty (immutable)

bytes to: **niemodyfikowalna (immutable) sekwencja bajtów**. Każdy bajt ma wartość **0–255**. Czyli nie tekst, nie liczby, **surowe dane binarne**

```
# Przykłady
bajty = b'hello'
bajty_od_listy = bytes([65, 66, 67]) # b'ABC'
```

```
# Zastosowania: dane binarne, pliki, sieć
dane_plikowe = b'\x89PNG\r\n\x1a\n' # nagłówek PNG
```

bytearray - tablice bajtów (mutable)

bytearray to: **modyfikowalna (mutable) sekwencja bajtów**. Każdy element: **liczba 0–255** („bajty do edycji”)

```
# Przykłady
modyfikowalne_bajty = bytearray(b'hello')
modyfikowalne_bajty[0] = 72 # H zamiast h -> b'Hello'
```

```
# Zastosowania: modyfikacja danych binarnych, bufory
bufor = bytearray(1024) # bufor 1KB
```

memoryview - widok pamięci

memoryview to: **widok na istniejący blok pamięci, w którym nie można kopiowania danych**. Działa na: bytes, bytearray, array, numpy itd. To **nie są dane** – to **okno na dane**.

```
# Przykłady
bajty = b'abcdef'
widok = memoryview(bajty)
print(widok[1:4].tobytes()) # b'bcd'
```

```
# Zastosowania: efektywna praca z dużymi danymi binarnymi
```

8. Specjalne typy

NoneType - wartość None

None to: **specjalna wartość oznaczająca „brak wartości”**

```
x = None
print(type(x)) # <class 'NoneType'>
```

None **to nie 0, nie "" i nie False**. Zastosowanie, kiedy coś **nie istnieje**, coś **jeszcze nie zostało ustawione**, funkcja **nic nie zwraca**, operacja **nie ma sensu**

```
# Przykłady
brak_wartosci = None
domyslna = None

# Zastosowania: reprezentacja braku wartości, domyślne argumenty
def znajdz_uzytkownika(id):
    if id in baza_danych:
        return baza_danych[id]
    return None
```

9. Typy z modułów

Decimal - liczby dziesiętne (dokładne)

```
from decimal import Decimal
```

```
# Przykłady
cena = Decimal('19.99')
podatek = Decimal('0.23')
kwota = cena * (1 + podatek)
```

Zastosowania: obliczenia finansowe (unikamy błędów zaokrągleń)

Fraction - ułamki zwykłe

```
from fractions import Fraction
```

```
# Przykłady
polowa = Fraction(1, 2)
trzecia = Fraction(1, 3)
suma = polowa + trzecia # Fraction(5, 6)
```

Zastosowania: obliczenia symboliczne, matematyka

Lekcja 2

Temat: Operatory, wyrażenia, if – elif – else, Pętle: For (z range i iterable), while; break, continue, else w pętlach w Python

1. Operatory Arytmetyczne

Używane do wykonywania podstawowych operacji matematycznych.

```
# Przykłady operatorów arytmetycznych
a = 10
b = 3

print(f"a = {a}, b = {b}")
print(f"Dodawanie: a + b = {a + b}")          # 13
print(f"Odejmowanie: a - b = {a - b}")        # 7
print(f"Mnożenie: a * b = {a * b}")           # 30
print(f"Dzielenie: a / b = {a / b}")           # 3.3333333333333335
print(f"Dzielenie całkowite: a // b = {a // b}") # 3
print(f"Reszta z dzielenia: a % b = {a % b}")  # 1
print(f"Potęgowanie: a ** b = {a ** b}")       # 1000

# Operatory przypisania z operacją
c = 5
c += 2 # równoważne: c = c + 2
print(f"c += 2: {c}") # 7

c *= 3 # c = c * 3
print(f"c *= 3: {c}") # 21
```

2. Operatory Porównania

Zwracają wartości logiczne (True/False) porównując wartości.

```
x = 10
y = 5
z = 10

print(f"x = {x}, y = {y}, z = {z}")
print(f"Równe: x == y -> {x == y}")          # False
print(f"Równe: x == z -> {x == z}")          # True
print(f"Różne: x != y -> {x != y}")          # True
print(f"Większe: x > y -> {x > y}")           # True
print(f"Mniejsze: x < y -> {x < y}")          # False
print(f"Większe lub równe: x >= z -> {x >= z}") # True
print(f"Mniejsze lub równe: y <= x -> {y <= x}") # True

# Porównania łańcuchów
print(f"'abc' == 'abc' -> {'abc' == 'abc'}") # True
print(f"'abc' != 'def' -> {'abc' != 'def'}") # True
"""

Každy znak ma swój numer Unicode:
ord('a') # 97
```

```
ord('b') # 98
"""
print(f"'a' < 'b' -> {'a' < 'b'}") # True (porównanie leksykograficzne)
```

3. Operatory Logiczne

Używane do łączenia wyrażeń logicznych.

Podstawowe operatory logiczne

```
a = True
b = False
```

```
print(f"a = {a}, b = {b}")
```

"""

Tabela prawdy - and

Operator and zwraca True tylko wtedy, gdy oba warunki są True.

a	b	a and b
True	True	True
True	False	False
False	True	False
False	False	False

"""

```
print(f"AND: a and b -> {a and b}") # False
```

"""

Tabela prawdy - or

Operator or zwraca True, jeśli chociaż jeden warunek jest True.

a	b	a or b
True	True	True
True	False	True
False	True	True
False	False	False

"""

```
print(f"OR: a or b -> {a or b}") # True
```

```
print(f"NOT: not a -> {not a}") # False
```

Praktyczne zastosowanie

```
wiek = 25
zarobki = 5000
```

Sprawdzanie wielu warunków

```
if wiek >= 18 and zarobki > 3000:
    print("Możesz wziąć kredyt") # Wykona się
```

Sprawdzanie czy wartość jest w zakresie

```
if 18 <= wiek <= 65:
    print("W wieku produkcyjnym") # Wykona się
```

Łączenie operatorów logicznych

```
x = 10
```

```
if x > 5 and x < 15 and x != 12:
    print("x spełnia wszystkie warunki") # Wykona się
```

4. Operatory Bitowe

```
# Operacje bitowe
'''
System binarny to dzielenie przez 2:
10 / 2 = 5 reszta 0
5 / 2 = 2 reszta 1
2 / 2 = 1 reszta 0
1 / 2 = 0 reszta 1
'''

a = 10 # 1010 w systemie binarnym bin(10), bin(10)[2:], format(10, 'b')
b = 6 # 0110 w systemie binarnym

print(f"a = {a} (bin: {bin(a)}), b = {b} (bin: {bin(b)})")
print(f"AND bitowe: a & b = {a & b}") # 2 (0010)
print(f"OR bitowe: a | b = {a | b}") # 14 (1110)
print(f"XOR bitowe: a ^ b = {a ^ b}") # 12 (1100)
print(f"NOT bitowe: ~a = {~a}") # -11 (dopełnienie)
'''

10 (dziesięć) = 1010 (binarnie)
Bo 1*8 + 0*4 + 1*2 + 0*1 = 10
<< 1 oznacza:
☞ przesuwamy wszystkie bity o jedno miejsce w lewo
☞ na końcu dopisujemy 0
10100
1*16 + 0*8 + 1*4 + 0*2 + 0*1 = 20

'''

print(f"Przesunięcie w lewo: a << 1 = {a << 1}") # 20 (10100)

'''

10 (dziesięć) = 1010 (binarnie)
Bo 1*8 + 0*4 + 1*2 + 0*1 = 10
>> 1 oznacza:
☞ przesuwamy wszystkie bity o jedno miejsce w prawo

1010 (10)
>> 1
-----
0101 (5)
0*8 + 1*4 + 0*2 + 1*1 = 5
'''

print(f"Przesunięcie w prawo: a >> 1 = {a >> 1}") # 5 (0101)

# Praktyczne zastosowanie - sprawdzanie parzystości
'''
print(f"7 = {bin(7)[2:]} ") # 111
print(f"1 = {bin(1)[2:]} ") # 1 czyli 001
Operacja AND (&)
```

Bit A	Bit B	Wynik
-------	-------	-------

1	1	1
1	0	0
0	1	0
0	0	0

```

    111 (7)
& 001 (1)
-----
    001
    ...
liczba = 7
if liczba & 1: # Ostatni bit = 1 dla liczb nieparzystych
    print(f"{liczba} jest nieparzysta")
else:
    print(f"{liczba} jest parzysta")

```

5. Priorytety Operatorów

Kolejność wykonywania działań.

```

# Przykłady priorytetów
wynik = 2 + 3 * 4 # Mnożenie ma wyższy priorytet
print(f"2 + 3 * 4 = {wynik}") # 14, nie 20

# Używanie nawiasów do zmiany kolejności
wynik = (2 + 3) * 4
print(f"(2 + 3) * 4 = {wynik}") # 20

# Przykład złożony
wynik = 5 + 2 ** 3 * 4 / 2 - 1
print(f"5 + 2 ** 3 * 4 / 2 - 1 = {wynik}")
# Kolejność: potęgowanie -> mnożenie/dzielenie -> dodawanie/odejmowanie

# Tabela priorytetów (od najwyższych do najniższych):
# 1. ** (potęgowanie)
# 2. +x, -x, ~x (jednoargumentowe)
# 3. *, /, //, %
# 4. +, - (dwuargumentowe)
# 5. <<, >> (przesunięcia bitowe)
# 6. & (AND bitowe)
# 7. ^ (XOR bitowe)
# 8. | (OR bitowe)
# 9. ==, !=, >, >=, <, <= (porównania)
# 10. not (logiczne NOT)
# 11. and (logiczne AND)
# 12. or (logiczne OR)

```

6. Wyrażenia Warunkowe

Instrukcje warunkowe i wyrażenia warunkowe (ternarne).

```

# Podstawowa instrukcja if-elif-else
liczba = 15

if liczba > 0:
    print("Liczba dodatnia")
elif liczba < 0:
    print("Liczba ujemna")
else:
    print("Zero")

# Zagnieżdżone warunki
wiek = 20
ma_prawo_jazdy = True

if wiek >= 18:
    if ma_prawo_jazdy:
        print("Możesz prowadzić samochód")
    else:
        print("Jesteś pełnoletni, ale nie masz prawa jazdy")
else:
    print("Jesteś niepełnoletni")

# Wyrażenie warunkowe (ternarne)
x = 10
y = 20

# Składnia: wartość_if_true if warunek else wartość_if_false
mniejsza = x if x < y else y
print(f"mniejsza liczba: {mniejsza}") # 10

# Praktyczny przykład - kategoria wiekowa
wiek = 25
kategoria = "dziecko" if wiek < 18 else ("dorosły" if wiek < 65 else "senior")
print(f"Wiek {wiek}: {kategoria}") # dorosły

# Operator walrus (:=) - dostępny od Python 3.8
# Pozwala na przypisanie wartości w ramach wyrażenia
if (n := len([1, 2, 3, 4])) > 3:
    print(f"Lista ma {n} elementów, czyli więcej niż 3")

```

if – elif – else

Używasz, gdy masz **kilka możliwych warunków**.

Przykład 1: Oceny szkolne

```

# System oceniania
punkty = 85

if punkty >= 90:
    ocena = "A"
    print("Celujący!")
elif punkty >= 80:
    ocena = "B"
    print("Bardzo dobry!")
elif punkty >= 70:

```

```

    ocena = "C"
    print("Dobry!")
elif punkty >= 60:
    ocena = "D"
    print("Dostateczny!")
else:
    ocena = "F"
    print("Niedostateczny!")

print(f"Za {punkty} punktów otrzymujesz ocenę: {ocena}")

```

Przykład 2: Weryfikacja wieku

```

wiek = int(input("Podaj swój wiek: "))

if wiek < 0:
    print("Wiek nie może być ujemny!")
elif wiek < 13:
    print("Jesteś dzieckiem")
elif wiek < 18:
    print("Jesteś nastolatkiem")
elif wiek < 65:
    print("Jesteś osobą dorosłą")
else:
    print("Jesteś seniorem")

# Sprawdzenie dodatkowych uprawnień
if wiek >= 18:
    print("Jesteś pełnoletni(a)")
    if wiek >= 21:
        print("Możesz legalnie pić alkohol w USA")
    if wiek >= 65:
        print("Przysługuje Ci emerytura")

```

2. Zagnieżdżone Warunki

Zagnieżdżone if pozwalają na tworzenie bardziej złożonych struktur decyzyjnych.

Przykład 1: System logowania

```

# Symulacja systemu logowania
poprawny_login = "admin"
poprawne_haslo = "secret123"
login_wprowadzony = "admin"
haslo_wprowadzone = "secret123"
kod_2fa = "7890"
if login_wprowadzony == poprawny_login:

```

```

print("Login poprawny")

if haslo_wprowadzone == poprawne_haslo:
    print("Hasło poprawne")

    if input("Podaj kod 2FA: ") == kod_2fa:
        print("Logowanie udane! Witamy w systemie.")
    else:
        print("Błędny kod 2FA!")
else:
    print("Błędne hasło!")
else:
    print("Błędny login!")

```

3. Ternary Operator (Operator Trójargumentowy)

Skrócona składnia dla prostych warunków.

Składnia: wartość_if_true if warunek else wartość_if_false

Przykład 1: Podstawowe użycie

```

# Tradycyjne podejście
liczba = 10
if liczba % 2 == 0:
    parzystosc = "parzysta"
else:
    parzystosc = "nieparzysta"

# To samo z ternary operator
liczba = 10
parzystosc = "parzysta" if liczba % 2 == 0 else "nieparzysta"
print(f"Liczba {liczba} jest {parzystosc}")

```

1. Pętla for z range()

Podstawowe użycie range()

```

# range(stop) - generuje liczby od 0 do stop-1
print("range(5):")
for i in range(5):
    print(i) # 0, 1, 2, 3, 4

# range(start, stop) - od start do stop-1

```

```

print("\nrange(2, 7):")
for i in range(2, 7):
    print(i) # 2, 3, 4, 5, 6

# range(start, stop, step) - z krokiem
print("\nrange(0, 10, 2):")
for i in range(0, 10, 2):
    print(i) # 0, 2, 4, 6, 8

# Ujemny krok - liczenie wstecz
print("\nrange(10, 0, -2):")
for i in range(10, 0, -2):
    print(i) # 10, 8, 6, 4, 2

```

2. Pętla for z iterable obiektami

Iteracja przez różne typy danych

```

# Listy
print("Iteracja przez listę:")
liczby = [10, 20, 30, 40, 50]
for liczba in liczby:
    print(liczba * 2)

# Napisy (stringi)
print("\nIteracja przez string:")
for litera in "Python":
    print(litera.upper(), end=" ")

# Krotki (tuple)
print("\n\nIteracja przez krotkę:")
wspolrzedne = (3, 5, 7, 9)
for wartosc in wspolrzedne:
    print(f"Wartość: {wartosc}")

# Słowniki
print("\nIteracja przez słownik:")
student = {'imie': 'Jan', 'wiek': 21, 'kierunek': 'Informatyka'}
for klucz in student:
    print(f"{klucz}: {student[klucz]}")

# Zbiory
print("\nIteracja przez zbiór:")
unikalne_liczby = {1, 3, 5, 3, 7, 1}
for liczba in unikalne_liczby:
    print(f"Unikalna: {liczba}")

```


3. Pętla while

Podstawowe zastosowania

```
# 1. Proste Liczenie
print("Liczenie do 5:")
licznik = 1

while licznik <= 5:
    print(licznik)
    licznik += 1

# 2. Symulacja gry - zgadywanie liczby
import random
print("\nGra w zgadywanie liczby:")

# losuje liczbę całkowitą (int) z zakresu od 1 do 50. Włącznie z 1 i 50
liczba_do_zgadnienia = random.randint(1, 50)
prob = 0
zgadniete = False

while not zgadniete and prob < 7:
    prob += 1
    strzal = int(input(f"Próba {prob}/7. Zgadnij liczbę (1-50): "))

    if strzal == liczba_do_zgadnienia:
        print(f"Brawo! Zgadłeś za {prob}. razem!")
        zgadniete = True
    elif strzal < liczba_do_zgadnienia:
        print("Za mało!")
    else:
        print("Za dużo!")

if not zgadniete:
    print(f"Przegrałeś! Liczba to: {liczba_do_zgadnienia}")

# 3. Kalkulator - menu z opcjami
print("\nProsty kalkulator (wpisz 'koniec' aby wyjść):")
while True:
    print("\n1. Dodawanie")
    print("2. Odejmowanie")
    print("3. Mnożenie")
    print("4. Dzielenie")
    print("5. Koniec")

    wybor = input("Wybierz opcję (1-5): ")
```

```

if wybor == '5' or wybor.lower() == 'koniec':
    print("Do widzenia!")
    break

if wybor in ['1', '2', '3', '4']:
    a = float(input("Podaj pierwszą liczbę: "))
    b = float(input("Podaj drugą liczbę: "))

    if wybor == '1':
        print(f"{a} + {b} = {a+b}")
    elif wybor == '2':
        print(f"{a} - {b} = {a-b}")
    elif wybor == '3':
        print(f"{a} × {b} = {a*b}")
    elif wybor == '4':
        if b != 0:
            print(f"{a} ÷ {b} = {a/b}")
        else:
            print("Błąd: Nie można dzielić przez zero!")
    else:
        print("Nieprawidłowy wybór!")

```

4. Instrukcje break, continue i else

break - natychmiastowe przerwanie pętli

```

# Szukanie pierwszej liczby podzielnej przez 7
print("Szukanie pierwszej liczby podzielnej przez 7:")
for i in range(1, 51):
    if i % 7 == 0:
        print(f"Znaleziono: {i}")
        break # przerywa pętlę natychmiast
    print(f"Sprawdzam {i}...")
print("Koniec wyszukiwania")

```

```

# Wyszukiwanie w liście
print("\nWyszukiwanie w liście książek:")
ksiazki = [
    "Harry Potter",
    "Władca Pierścieni",
    "Gra o Tron",
    "Mały Książę",
    "Hobbit"
]
szukana = "Mały Książę"

```

```

for ksiazka in ksiazki:
    if ksiazka == szukana:
        print(f"✓ Znaleziono: {ksiazka}")
        break
    print(f"Sprawdzam: {ksiazka}")
else:
    print("Książka nie została znaleziona")

# Break w pętli while
print("\nLimitowane próby logowania:")
haslo = "tajne123"
proby = 3

while proby > 0:
    wprowadzone = input(f"Pozostało prób: {proby}. Podaj hasło: ")
    if wprowadzone == haslo:
        print("Logowanie udane!")
        break
    else:
        print("Błędne hasło!")
        proby -= 1

if proby == 0:
    print("Konto zablokowane!")

```

continue - pominięcie bieżącej iteracji

```

# Wyświetlanie tylko liczb nieparzystych
print("Liczby nieparzyste 1-10:")
for i in range(1, 11):
    if i % 2 == 0: # jeśli parzysta
        continue # pominięcie reszty tej iteracji
    print(i)

# Przetwarzanie danych z pominięciem błędnych
print("\nPrzetwarzanie listy z błędnymi danymi:")
dane = [10, 0, 5, "błąd", 15, 0, 20]

for wartosc in dane:
    if not isinstance(wartosc, (int, float)):
        print(f"Pomijam nieprawidłowy typ: {wartosc}")
        continue

    if wartosc == 0:
        print("Pomijam zero (nie można dzielić)")
        continue

```

```

print(f"100 / {wartosc} = {100/wartosc:.2f}")

# Filtrowanie listy
print("\nFiltrowanie listy słów:")
slova = ["kot", "pies", "", "papuga", None, "chomik", " "]

for slowo in slova:
    if not slowo or not isinstance(slowo, str):
        continue

    slowo = slowo.strip()
    if not slowo: # puste po usunięciu spacji
        continue

    print(f"Długość słowa '{slowo}': {len(slowo)}")

```

else w pętlach -wykonuje się gdy pętla nie została przerwana przez break

```

# Sprawdzanie czy liczba jest pierwsza
def czy_pierwsza(n):
    if n < 2:
        return False

    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            print(f"{n} nie jest pierwsza (dzieli się przez {i})")
            break
    else:
        # Wykona się tylko jeśli pętla nie została przerwana przez break
        return True
    return False

print("Sprawdzanie liczb pierwszych:")
for liczba in [2, 3, 4, 5, 17, 21, 29]:
    if czy_pierwsza(liczba):
        print(f"{liczba} JEST liczbą pierwszą")
    else:
        print(f"{liczba} NIE JEST liczbą pierwszą")

# Szukanie elementu w liście
print("\nSzukanie wartości w liście:")
lista = [10, 20, 30, 40, 50]
szukana = 35

for element in lista:
    if element == szukana:

```

```

        print(f"Znaleziono {szukana}")
        break
    else:
        # Wykona się tylko jeśli nie znaleziono elementu
        print(f"Nie znaleziono {szukana} w liście")

# Walidacja danych wejściowych
print("\nWalidacja wprowadzonych liczb:")
while True:
    try:
        liczba = int(input("Podaj liczbę dodatnią: "))
        if liczba <= 0:
            print("Liczba musi być dodatnia!")
            continue
        break # jeśli doszliśmy tutaj, liczba jest poprawna
    except ValueError:
        print("To nie jest poprawna liczba!")
else:
    # W pętlach while, else wykonuje się jeśli nie było break
    # (ale tutaj break jest, więc else się nie wykona)
    print("Ta wiadomość się nie pojawi")

print(f"Wprowadzono poprawną liczbę: {liczba}")

```

Lekcja 3

Temat: Funkcje: Definiowanie i argumenty w Python

Podstawowa składnia:

```

def nazwa_funkcji(parametry):
    """Dokumentacja (opcjonalna)"""
    # ciało funkcji
    return wartość # opcjonalnie

```

Przykłady:

```

# Prosta metoda bez parametrów
def przywitaj():
    print("Cześć!")

```

```

# METODA z parametrem
def powitaj(imie):

```

```
print(f"Cześć, {imie}!")

# Funkcja zwracająca wartość
def dodaj(a, b):
    wynik = a + b
    return wynik
```

Argumenty funkcji

1. Argumenty pozycyjne - najprostsze, kolejność ma znaczenie:

```
def opisz_osobe(imie, wiek, miasto):
    print(f"{imie}, lat {wiek}, z {miasto}")

opisz_osobe("Anna", 25, "Warszawa") # OK
```

2. Argumenty nazwane (keyword arguments):

```
# Można podawać w dowolnej kolejności
opisz_osobe(miasto="Kraków", imie="Piotr", wiek=30)
```

3. Argumenty domyślne:

```
def powitaj(imie, jezyk="polski"):
    if jezyk == "polski":
        return f"Cześć, {imie}!"
    elif jezyk == "angielski":
        return f"Hello, {imie}!"
    else:
        return f"Witaj, {imie}!"

print(powitaj("Anna")) # używa domyślnego "polski"
print(powitaj("John", "angielski"))
```

⚠ **Ważne:** Argumenty domyślne muszą być na końcu listy parametrów:

```
# Poprawnie
def funkcja(a, b=10): ...
```

```
# Błędnie - parametr domyślny przed wymaganym
def funkcja(a=10, b): ... # SyntaxError
```

4. *args - dowolna liczba argumentów pozycyjnych:

przekazywane **bez nazw**. Nie piszemy `sumuj_wszystko(a=10, b=20)`

```
def sumuj_wszystko(*liczby):
    """Przyjmuje dowolną liczbę argumentów"""
    return sum(liczby)

print(sumuj_wszystko(1, 2, 3))      # 6
print(sumuj_wszystko(10, 20))      # 30
print(sumuj_wszystko(1, 2, 3, 4, 5)) # 15
```

5. **kwargs - dowolna liczba argumentów nazwanych:

```
def pokaz_dane(**dane):
    """Przyjmuje dowolną liczbę argumentów nazwanych"""
    for klucz, wartosc in dane.items():
        print(f"{klucz}: {wartosc}")

pokaz_dane(imie="Anna", wiek=25, miasto="Warszawa")
# Wyświetli:
# imie: Anna
# wiek: 25
# miasto: Warszawa
```

6. Łączenie różnych typów argumentów:

```
def skomplikowana_metoda(a, b, *args, opcja=None, **kwargs):
    print(f"a={a}, b={b}") # pamiętać o wcięciach
    print(f"args={args}")
    print(f"opcja={opcja}")
    print(f"kwargs={kwargs}")

# Przykład wywołania:
skomplikowana_funkcja(1, 2, 3, 4, 5, opcja="test", x=100, y=200)
```

Kolejność parametrów musi być:

1. Argumenty pozycyjne
2. *args
3. Argumenty domyślne/nazwane
4. **kwargs

Lekcja 4

Temat: lambda. Zasięg zmiennych (scope) Łańcuchy znaków: Stringi, formatowanie (f-strings, format()), metody (split, join, replace) w Python

lambda pozwala tworzyć **małe, anonimowe funkcje** (czyli funkcje bez nazwy)
Czyli zamiast:

```
def dodaj(a, b):  
    wynik = a + b  
    return wynik
```

Skrótowa wersja w lambda:

```
lambda a, b: a + b
```

Składnia:

```
lambda argumenty: wyrażenie
```

Istotne:

- Zawierać może **tylko jedno wyrażenie**
- Nie możesz używać w niej instrukcji typu **if, for, while, return**

Zasięg to **obszar kodu**, w którym dana nazwa (zmienna, funkcja, klasa) jest widoczna i można ją zastosować.

Python stosuje **regułę LEGB** – szuka nazwy dokładnie w tej kolejności:

Litera	Nazwa	Gdzie szuka
--------	-------	-------------

L	Local	wewnątrz aktualnej funkcji / metody
E	Enclosing	w funkcjach otaczających (nested functions)
G	Global	na poziomie modułu (poza wszystkimi funkcjami)
B	Built-in	wbudowane nazwy (print, len, int, range...)

2. Rodzaje zasięgu – przykłady

a) Local (lokalny)

```
def metoda():
    x = 10      # ← zmienna lokalna
    print(x)    # 10

metoda()
# print(x)      # NameError – x nie istnieje poza funkcją
```

b) Global (globalny)

```
x = 5
def metoda():
    print(x)

metoda()
```

c) Enclosing (otaczający) + nonlocal

```
def zewnetrzna():
    x = 10      # enclosing scope

    def wewnetrzna():
        nonlocal x      # Nie tworzy nowej zmiennej lokalnej. Użyje zmiennej z najbliższego
        # zewnętrznego scope (enclosing scope).
        x = 20

    wewnetrzna()
    print(x)      # 20

zewnetrzna()
```

Bez nonlocal:

```
def zewnetrzna():
    x = 10

    def wewnetrzna():
        x = 20      # tworzy nową zmienną LOKALNĄ!
        print(x)    # 20

    wewnetrzna()
    print(x)        # 10 (oryginalna nie zmieniła się)

zewnetrzna()
```

d) Built-in

```
print(len([1, 2, 3])) # len i print są z built-in scope
```

built-in scope to **najbardziej zewnętrzny zakres**, który zawsze istnieje. Zawiera funkcje i typy wbudowane w Pythona, np.:

- print
- len
- int
- str
- list
- dict
- sum
- min
- max

3. Modyfikowanie zmiennych globalnych – global

```
licznik = 0
```

```
def zwieksz():
    global licznik    # bez tego dostalibyśmy UnboundLocalError: cannot access local
                     # variable 'licznik' where it is not associated with a value
```

```
licznik += 1
```

```
zwiększ()
```

```
zwiększ()
```

```
print(licznik)          # 2
```

Przykład z lambda (klasyczna pułapka):

```
funcs = []
```

```
for i in range(3):
```

```
    funcs.append(lambda: i)
```

```
print([f() for f in funcs])
```

Poprawnie:

```
funcs = []
```

```
for i in range(3):
```

```
    funcs.append(lambda x=i: x) # domyślny argument "zamraża" wartość
```

```
print([f() for f in funcs])
```

globals() i locals() – funkcje do inspekcji aktualnego scope

1. globals() – zwraca słownik wszystkich zmiennych globalnych (poziom modułu)

plik: test_scope.py

```
a = 10
```

```
b = "hello"
```

```
c = [1, 2, 3]
```

```
print("== globals() na poziomie modułu ==")
```

```
print(globals())
```

== globals() na poziomie modułu ==

{

'a': 10,

'b': 'hello',

'c': [1, 2, 3],

'__name__': '__main__',

```
# '_builtins_': <module 'builtins' ...>,  
# ...  
# }
```

2. locals() – zwraca słownik zmiennych lokalnych aktualnego scope

Przykład wewnątrz funkcji:

```
a = 90  
def metoda_testowa():  
    x = 42  
    y = "Python"  
    z = [10, 20, 30]
```

```
print("=== locals() wewnątrz funkcji ===")  
print(locals())
```

```
# === locals() wewnątrz funkcji ===  
# {'x': 42, 'y': 'Python', 'z': [10, 20, 30]}
```

```
metoda_testowa()
```

```
global_var = "globalna"
```

```
def test():  
    local_var = "lokalna"
```

```
print("globals() zawiera global_var:", "global_var" in globals()) # True  
print("locals() zawiera local_var: ", "local_var" in locals()) # True  
print("locals() zawiera global_var:", "global_var" in locals()) # False
```

```
test()
```

Modyfikacja przez słownik:

```
nowa = 123
```

```
def demo():  
    x = 100  
    locals()["x"] = 999
```

```
print(x)
```

```
globals()["nowa"] = 42
```

```
print(nowa)
```

```
demo()
```

ŁAŃCUCHY ZNAKÓW (STRINGS)

Stringi to sekwencje znaków, które mogą zawierać litery, cyfry, symbole i spacje.

```
# Różne sposoby tworzenia stringów
```

```
imie = "Anna"
```

```
nazwisko = 'Kowalska'
```

```
zdanie = """To jest
```

```
wielolinijkowy
```

```
tekst"""
```

```
print(imie)      # Anna
```

```
print(nazwisko)  # Kowalska
```

```
print(zdanie)    # To jest\nwielolinijkowy\ntekst
```

```
# Podstawowe operacje na stringach
```

```
powitanie = "Cześć, " + imie + "!" # konkatencja (łączenie)
```

```
print(powitanie) # Cześć, Anna!
```

```
powtorzenie = "git" * 3 # powielanie
```

```
print(powtorzenie) # gitgitgit
```

```
# Indeksowanie i wycinanie (slicing)
```

```
tekst = "Python"
```

```
print(tekst[0])    # P (pierwszy znak)
```

```
print(tekst[-1])   # n (ostatni znak)
```

```
print(tekst[1:4])  # yth (od indeksu 1 do 3)
```

```
print(tekst[:3])    # Pyt (od początku do indeksu 2)
print(tekst[3:])    # hon (od indeksu 3 do końca)
```

Metoda format()

```
imie = "Piotr"
wiek = 35

# Pozycyjnie
print("Nazywam się {} i mam {} lat.".format(imie, wiek))

# Z indeksami
print("Nazywam się {0} i mam {1} lat. {0} lubi
programować.".format(imie, wiek))

# Z nazwami
print("Nazywam się {name} i mam {age} lat.".format(name=imie,
age=wiek))

# Formatowanie liczb
liczba = 3.14159
print("Liczba Pi to {:.2f}".format(liczba))
```

METODY STRINGÓW

split() - dzielenie stringa na listę

```
# Podstawowe split()
zdanie = "Python jest świetny"
slova = zdanie.split()
print(slova)  # ['Python', 'jest', 'świetny']
```

```

# Split z własnym separatorem
data = "2024-01-15"
czesc_daty = data.split("-")
print(czesc_daty) # ['2024', '01', '15']

# Split z limitem podziałów
tekst = "jabłko,gruszka,banan,wiśnia"
owoce = tekst.split(",", 2)
print(owoce) # ['jabłko', 'gruszka', 'banan,wiśnia']

# Praktyczny przykład - parsowanie danych
dane_logowania = "user:haslo123"
login, haslo = dane_logowania.split(":")
print(f"Login: {login}, Hasło: {haslo}")

```

join() - łączenie listy w string

```

# Podstawowe join()
owoce = ['jabłko', 'gruszka', 'banan']
tekst = ", ".join(owoce)
print(tekst) # jabłko, gruszka, banan

# Różne separatory
slova = ['Python', 'jest', 'fajny']
print(" ".join(slova)) # Python jest fajny
print("-".join(slova)) # Python-jest-fajny
print("".join(slova)) # Pythonjestfajny

# Łączenie z transformacją
liczby = [1, 2, 3, 4, 5]
# Najpierw musimy zamienić liczby na stringi
tekst = ", ".join(str(x) for x in liczby)
print(tekst) # 1, 2, 3, 4, 5

```

```
# Praktyczny przykład - tworzenie ścieżki
katalogi = ['home', 'user', 'dokumenty']
sciezka = '/'.join(katalogi)
print(sciezka) # home/user/dokumenty
```

replace() - zamiana fragmentów stringa

```
tekst = "Lubię koty. Koty są fajne."
nowy_tekst = tekst.replace("koty", "psy")
print(nowy_tekst)

tekst = "Lubię koty. Koty są fajne."
nowy_tekst = tekst.replace("koty", "psy").replace("Koty", "Psy")
print(nowy_tekst)

tekst = "jabłko jabłko jabłko"
print(tekst.replace("jabłko", "gruszka", 2))
tekst = "Hello, World!!!"
czysty = tekst.replace(",", "").replace("!", "")
print(czysty)

numer_telefonu = "+48 123-456-789"
czysty_numer = numer_telefonu.replace("+48 ", "").replace("-", "")
print(czysty_numer)
```

Przykład 1: Czyszczenie danych wejściowych

```
def czyszczenie_dane(tekst):
    tekst = tekst.strip()
    while " " in tekst:
        tekst = tekst.replace(" ", "")
    tekst = tekst.lower()
```



```
    return tekst

dane_wejsciowe = "  To  Jest  Tekst  "
czyste = czysc_dane(dane_wejsciowe)
print(czyste)
```

Przykład 2: Generowanie raportów

```
def generuj_raport(studenci):
    linie = []
    for imie, ocena in studenci:
        linie.append(f"{imie:10} - {ocena}")

    return "\n".join(linie)

studenci = [("Anna", 5), ("Jan", 4), ("Katarzyna", 4.5)]
print(generuj_raport(studenci))

# Wynik:
# Anna      - 5
# Jan       - 4
# Katarzyna - 4.5
```

Lekcja 5

Temat: Zastosowania programowania obiektowego (OOP) w Pythonie. Wyjaśnienie kluczowych pojęć OOP: Klasa (Class), Obiekt (Object), Metody (Methods), Atrybuty (Attributes), init (konstruktor)

Programowanie obiektowe (OOP) w Pythonie to **paradygmat** programowania, który pozwala na **organizowanie kodu wokół "obiektów"** zamiast funkcji i procedur. Jest to jedna z kluczowych cech Pythona, czyniąca go elastycznym i łatwym w utrzymaniu dla dużych projektów.

Kluczowe pojęcia OOP

1. Klasa (Class)

Klasa to szablon lub blueprint **definiujący strukturę i zachowanie obiektów**.

Definiuje, jakie atrybuty (dane) i metody (funkcje) będą miały obiekty stworzone na jej podstawie. Klasa sama w sobie nie przechowuje danych – to robią jej instancje (obiekty).

Przykład:

```
class Pies:  
    pass
```

```
moj_pies = Pies()  
print(type(moj_pies))
```

2. Obiekt (Object)

Obiekt to instancja (konkretny egzemplarz) **klasy**. **Obiekt ma własne atrybuty i może wywoływać metody zdefiniowane w klasie**. Tworzysz obiekt wywołując klasę jak funkcję (np. Klasa()). Obiekty są unikalne, nawet jeśli pochodzą z tej samej klasy.

Przykład:

```
class Pies:  
    def szczekaj(self):  
        print("Hau hau!")
```

```
moj_pies = Pies()  
inny_pies = Pies()  
  
moj_pies.szczekaj()  
print(moj_pies == inny_pies)
```

2. a) self

self to odniesienie (referencja) do bieżącej instancji klasy (konkretnego obiektu). To sposób, w jaki obiekt "widzi samego siebie" i może operować na swoich własnych danych.

self w metodach klasy

```
class Osoba:  
    def przedstaw_sie(self):
```

```

        print(f"Jestem obiektem: {self}")
        print(f"Mój typ to: {type(self)}")
osoba1 = Osoba()
osoba2 = Osoba()
print("Wywołanie dla osoby1:")
osoba1.przedstaw_sie() # self = osoba1
print("\nWywołanie dla osoby2:")
osoba2.przedstaw_sie() # self = osoba2

```

Wynik:

Wywołanie dla osoby1: Jestem obiektem: <__main__.Osoba object at 0x7c84a9049be0>
Mój typ to: <class '__main__.Osoba'>
Wywołanie dla osoby2: Jestem obiektem: <__main__.Osoba object at 0x7c84a9049c10>
Mój typ to: <class '__main__.Osoba'>

self w __init__

Metoda `__init__` jest wywoływana automatycznie podczas tworzenia obiektu. `self` w `__init__` odnosi się do **nowo tworzonego obiektu**:

```

class Pracownik:
    def __init__(self, imie, pensja):
        print(f"1. Tworzę obiekt: {self}")
        print(f"2. Przypisuję imię '{imie}' do self.imie")

        self.imie = imie
        self.pensja = pensja

        print(f"3. Obiekt po inicjalizacji: imię={self.imie}, pensja={self.pensja}")
        print(f"4. ID obiektu: {id(self)}")

p = Pracownik("Anna", 5000)
print(f"\nUtworzony obiekt: {p}")
print(f"ID obiektu z zewnątrz: {id(p)}")

```

Wynik:

1. Tworzę obiekt: <__main__.Pracownik object at 0x7fa10867db50>
2. Przypisuję imię 'Anna' do self.imie
3. Obiekt po inicjalizacji: imię=Anna, pensja=5000
4. ID obiektu: 140329607486288

Utworzony obiekt: <__main__.Pracownik object at 0x7fa10867db50>
ID obiektu z zewnątrz: 140329607486288

self odwołuje się do atrybutów obiektu

```
class Samochod:
    def __init__(self, marka):

        print(f"{self}") # <__main__.Samochod object at 0x7b916db45be0>

        self.marka = marka
        self.predkosc = 0

    def przyspiesz(self, wartosc):
        # self.predkosc - odwołanie do atrybutu
        self.predkosc += wartosc
        print(f"{self.marka}: jadę z prędkością {self.predkosc} km/h")

    def hamuj(self):
        # self.predkosc - modyfikacja atrybutu
        self.predkosc = 0
        print(f"{self.marka}: zatrzymałem się")

auto = Samochod("Toyota")
auto.przyspiesz(50) # self to auto
auto.hamuj()        # self to auto
```

self pozwala odróżnić atrybuty obiektu od zmiennych lokalnych

```
class Przyklad:
    def __init__(self, wartosc):
        # wartosc - to parametr (zmienna lokalna)
        # self.wartosc - to atrybut obiektu
        self.wartosc = wartosc

    def pokaz(self, wartosc):
        # wartosc - to parametr metody
        # self.wartosc - to atrybut obiektu
        print(f"Parametr metody: {wartosc}")
        print(f"Atrybut obiektu: {self.wartosc}")

ob = Przyklad(100)
ob.pokaz(50) # Parametr: 50, Atrybut: 100
```

Python automatycznie przekazuje obiekt jako pierwszy argument przy wywołaniu metody:

```
python
```

```
class Test:
    def metoda(self, arg1, arg2):
        print(f"self: {self}")
        print(f"arg1: {arg1}, arg2: {arg2}")

t = Test()

# Te dwa wywołania są RÓWNOWAŻNE:
t.metoda(10, 20)          # Python automatycznie wstawia self
Test.metoda(t, 10, 20)    # Ręczne przekazanie obiektu jako self
```

3. Metody (Methods)

Metody to funkcje zdefiniowane wewnątrz klasy, które operują na obiektach.

Pierwszy parametr metody to zawsze self (odwołuje się do bieżącego obiektu).

Metody pozwalają na interakcję z obiektem. Mogą modyfikować atrybuty lub zwracać wartości. Są wywoływane na obiekcie: obiekt.metoda().

```
class Samochod:
    def jedz(self, predkosc):
        print(f"Samochód jedzie z prędkością {predkosc} km/h.")

    def zatrzymaj(self):
        print("Samochód się zatrzymał.")

moj_samochod = Samochod() # Obiekt
moj_samochod.jedz(100)    # Wyjście: Samochód jedzie z prędkością 100 km/h.
moj_samochod.zatrzymaj()  # Wyjście: Samochód się zatrzymał.
```

4. Atrybuty (Attributes)

Atrybuty to zmienne związane z klasą lub obiektem. Mogą być atrybutami klasy (wspólne dla wszystkich obiektów) lub **instancji** (unikalne dla każdego obiektu).

Atrybuty przechowują dane. Dostęp do nich: obiekt.atrybut. Mogą być ustawiane w `__init__` lub bezpośrednio.

```
class Osoba:
    gatunek = "Człowiek" # Atrybut klasy (wspólny)

    def __init__(self, imie): # Więcej o __init__ poniżej
        self.imie = imie      # Atrybut instancji (unikalny)

jan = Osoba("Jan")           # Obiekt
print(jan.imie)              # Wyjście: Jan
print(jan.gatunek)           # Wyjście: Człowiek
```

```
anna = Osoba("Anna")
print(anna.imie)           # Wyjście: Anna
print(anna.gatunek)        # Wyjście: Człowiek (wspólny)
```

5. init (Konstruktor)

`__init__` to specjalna metoda (konstruktor), która jest wywoływana automatycznie przy tworzeniu obiektu. Służy do inicjalizacji atrybutów obiektu. Nie zwraca wartości, ale ustawia początkowe stany. Pierwszy parametr to `self`. Możesz podać argumenty przy tworzeniu obiektu.

```
class Ksiazka:
    def __init__(self, tytuł, autor, rok=2023): # Konstruktor z domyślnym
parametrem
        self.tytuł = tytuł                    # Inicjalizacja atrybutów
        self.autor = autor
        self.rok = rok
```

```
    def opis(self): # Funkcja używająca atrybutów
        return f"{self.tytuł} autorstwa {self.autor} ({self.rok})"
```

```
moja_ksiazka = Ksiazka("Python dla początkujących", "Jan Kowalski") #
Tworzenie z argumentami
print(moja_ksiazka.opis()) # Wyjście: Python dla początkujących autorstwa Jan
Kowalski (2023)
```

```
inna_ksiazka = Ksiazka("Zaawansowany Python", "Anna Nowak", 2024)
print(inna_ksiazka.opis()) # Wyjście: Zaawansowany Python autorstwa Anna Nowak
(2024)
```

Lekcja 6

Temat: Kartkówka i przypisanie projektów

Lekcja 7

Temat: Działanie dziedziczenia klas w Pythonie: Klasy bazowe i pochodne, `super()`, wielokrotne dziedziczenie.

Dokumentacja Python odnośnie dziedziczenia:

<https://docs.python.org/pl/dev/tutorial/classes.html#inheritance>

Dziedziczenie w Pythonie to mechanizm programowania obiektowego (OOP), który pozwala na tworzenie nowej klasy na podstawie istniejącej klasy.

Nowa klasa (pochodna) "dziedziczy" atrybuty i metody z klasy bazowej, co oznacza, że może je wykorzystywać bez potrzeby ich ponownego definiowania. Działanie polega na tym, że klasa pochodna rozszerza lub modyfikuje funkcjonalność klasy bazowej, **tworząc hierarchię klas**. Gdy stworzysz instancję klasy pochodnej, Python najpierw szuka atrybutów i metod w tej klasie, a jeśli ich nie znajdzie, **przeszukuje klasy bazowe w kolejności określonej przez metodę rozdzielczości (Method Resolution Order, MRO)**. To umożliwia ponowne użycie kodu i budowanie bardziej złożonych struktur.

Definicje kluczowych pojęć

- ❑ **Klasa bazowa (base class lub parent class):** To klasa, z której dziedziczą inne klasy. Zawiera wspólne atrybuty i metody, które mogą być udostępniane klasom pochodnym. W Pythonie klasa bazowa jest definiowana normalnie, a dziedziczenie odbywa się poprzez podanie jej nazwy w nawiasach przy definicji klasy pochodnej.
- ❑ **Klasa pochodna (derived class lub child class):** To klasa, która dziedziczy z klasy bazowej. Może dodawać nowe atrybuty i metody lub nadpisywać (override) te z klasy bazowej, dostosowując je do swoich potrzeb.
- ❑ **super():** To wbudowana funkcja Pythona, która pozwala na wywoływanie metod klasy bazowej z poziomu klasy pochodnej. Jest szczególnie przydatna w konstruktorach (`__init__`) lub przy nadpisywaniu metod, aby uniknąć powtarzania kodu. `super()` automatycznie odnosi się do klasy bazowej według MRO, co jest bezpieczniejsze niż bezpośrednie wywoływanie nazwy klasy bazowej (zwłaszcza przy wielokrotnym dziedziczeniu).
- ❑ **Wielokrotne dziedziczenie:** To mechanizm, w którym klasa pochodna dziedziczy z więcej niż jednej klasy bazowej. W Pythonie jest to wspierane, ale wymaga ostrożności ze względu na potencjalne konflikty (np. diamentowy problem dziedziczenia). Kolejność dziedziczenia określa MRO, które Python rozwiązuje za pomocą algorytmu C3 linearization.

1. Podstawowe dziedziczenie (klasa bazowa i pochodna)

```
class Animal:
    def __init__(self, name):
        self.name = name
```

```

    def speak(self):
        return f"{self.name} makes a sound."

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks."

dog = Dog("Rex")
print(dog.speak())

```

2. Użycie super() w konstruktorze

```

class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed

    def info(self):
        return f"{self.name} is a {self.breed}."

dog = Dog("Rex", "Labrador")
print(dog.info())

```

3. Wielokrotne dziedziczenie

```

class Flyer:
    def fly(self):
        return "Flying high!"

class Swimmer:
    def swim(self):
        return "Swimming fast!"

class Duck(Flyer, Swimmer):
    def quack(self):
        return "Quack!"

duck = Duck()
print(duck.fly())
print(duck.swim())

```



```
print(duck.quack())
print(Duck.mro())
```

`print(Duck.mro())` wypisuje **kolejność rozwiązywania metod (MRO – Method Resolution Order)** dla klasy Duck.

Wynik `Duck.mro()`

[Duck, Flyer, Swimmer, object]

[<class '__main__.Duck'>, <class '__main__.Flyer'>, <class '__main__.Swimmer'>, <class 'object'>]

Czyli Python będzie szukał metod w tej kolejności:

1. Duck
2. Flyer
3. Swimmer
4. object (bazowa klasa wszystkich klas w Pythonie)

Lekcja 8

Temat: Słowniki (Dictionaries) i zbiory (Sets).

Klasa abstrakcyjna

1. Słowniki (dict) – pary klucz-wartość

Słownik to struktura danych przechowująca pary **klucz-wartość**. Klucze muszą być unikalne i niemutowalne (np. string, int, tuple, frozenset). Wartości mogą być dowolnego typu (nawet inny słownik).

1. Tworzenie słownika

```
osoba = {
    "imie": "Anna",
    "wiek": 28,
    "miasto": "Kraków"
}
```

2. Dodawanie i modyfikacja pojedynczych elementów

```
osoba["email"] = "anna@example.com"    # nowy klucz
osoba["wiek"] = 29                     # nadpisanie istniejącej wartości
```

3. Aktualizacja wieloma kluczami naraz (dwie metody)

```
osoba.update({"telefon": "123456789", "wiek": 30})
```

```
osoba.update(imie="Anna Kowalska", stanowisko="programista")
```

```
print("=== Aktualny słownik ===")
print(osoba)
print() # pusta linia dla czytelności
```

```
print("1. Najczęściej używane – klucz i wartość")
for klucz, wartosc in osoba.items():
    print(f"{klucz} -> {wartosc}")
```

```
print("\n 2. Tylko klucze")
for klucz in osoba: # lub osoba.keys()
    print(klucz)
```

```
print("\n 3. Tylko wartości")
for wartosc in osoba.values():
    print(wartosc)
```

```
print("\n 4. Ładniejsze formatowanie (f-string)")
for klucz, wartosc in osoba.items():
    print(f"{klucz:12} : {wartosc}")
```

Przykład zastosowania:

```
tekst = "python jest super python jest fajny python"
licznik = {}
for slowo in tekst.split():
    licznik[slowo] = licznik.get(slowo, 0) + 1
print(licznik)
```

```
studenci = {
    "12345": {"imie": "Jan", "ocena": 4.5},
    "67890": {"imie": "Ola", "ocena": 5.0}
}
studenci["12345"]["ocena"] = 5.0
```

```
config = {
    "debug": True,
    "api_url": "https://api.example.com",
    "timeout": 30,
    "retry": 3
}
config.update({"debug": False, "timeout": 60})
```

2. Zbiory (set) – unikalne elementy

Zbiór to **nieuporządkowana** kolekcja **unikalnych** elementów. Nie ma indeksów, nie można powtarzać elementów.

Tworzenie i operacje add / update

```
owoce = {"jabłko", "banan", "pomarańcza"}
pusty = set()
owoce.add("kiwi")
owoce.update(["truskawka", "banan", "malina"])  # "banan" już istnieje →
pominięty
print(owoce)
```

Najważniejsze operacje zbiorów

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

A | B  # suma (union) → {1,2,3,4,5,6} A.union(B)
A & B  # przecięcie (intersection) → {3,4}
A - B  # różnica A.difference(B) → {1,2}
A ^ B  # symetryczna różnica A.symmetric_difference(B) → {1,2,5,6}

A.add(99) # dodaje element w dowolnym miejscu
A.update([100, 101]) # dodaje wiele elementów
A.remove(1)          # błąd jeśli nie ma
A.discard(999)       # Usuwa element. Nie rzuca błędu, jak element nie istnieje
x = A.pop()          # usuwa losowy element i zwraca usuwany element
```

Przykłady zastosowań w praktyce

```
lista = [1, 2, 2, 3, 4, 4, 5]
unikalne = list(set(lista))

zablokowane_ip = {"192.168.1.1", "10.0.0.5", "172.16.0.1"}
if "192.168.1.1" in zablokowane_ip:
    print("Zablokuj!")

tagi_uzytkownika1 = {"python", "django", "machine-learning"}
tagi_uzytkownika2 = {"python", "flask", "data-science"}

wspolne = tagi_uzytkownika1 & tagi_uzytkownika2  # {'python'}
```

```
wszystkie = tagi_uzytkownika1 | tagi_uzytkownika2
tylko1 = tagi_uzytkownika1 - tagi_uzytkownika2
```

Mały kompletny przykład łączący obie struktury

```
posty = [
    {"tagi": ["python", "django"]},
    {"tagi": ["python", "flask", "api"]},
    {"tagi": ["django", "python"]}
]

wszystkie_tagi = set()
licznik_tagow = {}
for post in posty:
    for tag in post["tagi"]:
        wszystkie_tagi.add(tag)
        licznik_tagow[tag] = licznik_tagow.get(tag, 0) + 1

print("Unikalne tagi:", wszystkie_tagi)
print("Popularność:", licznik_tagow)
```

3. Klasa abstrakcyjna - **to trzeba nadrobić**

Klasa abstrakcyjna to klasa, której **nie można instancjonować** (nie można tworzyć obiektów bezpośrednio z tej klasy). Służy jako **szablon** dla innych klas - definiuje metody, które klasy pochodne **muszą** zaimplementować.

- Python udostępnia moduł **abc** (Abstract Base Classes) do tworzenia klas abstrakcyjnych.
- Metody abstrakcyjne oznaczamy dekoratorem `@abstractmethod`.
- **Nie można utworzyć instancji** klasy abstrakcyjnej, jeśli ma choć jedną metodę abstrakcyjną.
- Każda klasa dziedzicząca **musi** zaimplementować wszystkie metody abstrakcyjne – inaczej dostaniesz błąd przy próbie utworzenia obiektu.
- Możesz dodawać zwykłe metody (nieabstrakcyjne) i pola – one są dziedziczone normalnie.

Przykład:

```
from abc import ABC, abstractmethod
from datetime import datetime
```

```

class PaymentProcessor(ABC):

    def __init__(self, api_key: str):
        self.api_key = api_key
        self.transaction_log = []

    @abstractmethod
    def process_payment(self, amount: float, currency: str) -> str:
        pass

    @abstractmethod
    def refund(self, transaction_id: str) -> bool:
        pass

    def log_transaction(self, transaction_id: str, amount: float):
        self.transaction_log.append({
            "id": transaction_id,
            "amount": amount,
            "timestamp": datetime.now()
        })
        print(f"Zalogowano transakcję: {transaction_id}")


class StripeProcessor(PaymentProcessor):
    def process_payment(self, amount: float, currency: str) -> str:

        transaction_id = f"stripe_{int(datetime.now().timestamp())}"
        print(f": Płatność {amount} {currency} przyjęta")
        self.log_transaction(transaction_id, amount)
        return transaction_id

    def refund(self, transaction_id: str) -> bool:
        print(f": Stripe: Zwrot dla {transaction_id}")
        return True


class PayPalProcessor(PaymentProcessor):
    def process_payment(self, amount: float, currency: str) -> str:
        transaction_id = f"paypal_{int(datetime.now().timestamp())}"
        print(f"PayPal: Płatność {amount} {currency} przyjęta")
        self.log_transaction(transaction_id, amount)

```

```

    return transaction_id

def refund(self, transaction_id: str) -> bool:
    print(f"PayPal: Zwrot dla {transaction_id}")
    return True

# processor = PaymentProcessor("abc123") # TypeError: Can't instantiate
# abstract class

stripe = StripeProcessor("sk_test_12345")
paypal = PayPalProcessor("paypal_key_999")

trans_id1 = stripe.process_payment(99.99, "PLN")
trans_id2 = paypal.process_payment(49.50, "EUR")

stripe.refund(trans_id1)

```

Najczęstsze zastosowania w prawdziwych projektach:

Zastosowanie	Przykład klasy abstrakcyjnej
System płatności	PaymentProcessor (jak powyżej)
Różne formaty eksportu	ReportExporter (PDF, Excel, CSV)
Silniki AI / modele ML	AIModel (z metodą predict())
Sterowniki urządzeń	DeviceDriver (USB, Bluetooth)
Pluginy w aplikacji	Plugin (metoda execute())
Testy jednostkowe (mocki)	Abstrakcyjne repozytoria

Pokażę Ci **dwie nowoczesne wersje** klasy abstrakcyjnej:

1. Klasyczna wersja z **właściwościami abstrakcyjnymi** (@property + @abstractmethod)
2. Najnowocześniejsze podejście: typing.Protocol (Python 3.8+) – bez dziedziczenia!

1. Wersja z właściwościami abstrakcyjnymi (zalecana w klasach ABC)

```

from abc import ABC, abstractmethod
from datetime import datetime

class PaymentProcessor(ABC):
    """Klasa abstrakcyjna z właściwościami (property)
    @property pozwala używać metody jak pola (atrybutu).
    """

```

```

    """

    @property
    @abstractmethod
    def provider_name(self) -> str:
        """Nazwa dostawcy płatności (np. 'Stripe', 'PayPal')"""
        pass

    @abstractmethod
    def process_payment(self, amount: float, currency: str) -> str:
        pass

    def log(self, message: str):
        print(f"[{self.provider_name}] {message}")

class StripeProcessor(PaymentProcessor):
    @property
    def provider_name(self) -> str:
        return "Stripe"

    def process_payment(self, amount: float, currency: str) -> str:
        self.log(f"Przetwarzam płatność {amount} {currency}")
        return f"stripe_tx_{int(datetime.now().timestamp())}"

class PayPalProcessor(PaymentProcessor):
    @property
    def provider_name(self) -> str:
        return "PayPal"

    def process_payment(self, amount: float, currency: str) -> str:
        self.log(f"Przetwarzam płatność {amount} {currency}")
        return f"paypal_tx_{int(datetime.now().timestamp())}"

stripe = StripeProcessor()
paypal = PayPalProcessor()

print(stripe.provider_name)                                # Stripe
print(paypal.process_payment(99.99, "PLN"))

```

2. Najnowocześniejsze podejście: typing.Protocol (Python 3.8+)

To już **nie wymaga dziedziczenia**! Działa na zasadzie „strukturalnego typowania” (duck typing na sterydach).

```
from typing import Protocol
from datetime import datetime

class PaymentProcessor(Protocol):
    """Nie dziedziczymy – tylko deklarujemy interfejs!"""

    @property
    def provider_name(self) -> str: ...

    def process_payment(self, amount: float, currency: str) -> str: ...


class StripeProcessor:
    @property
    def provider_name(self) -> str:
        return "Stripe"

    def process_payment(self, amount: float, currency: str) -> str:
        print(f"[{self.provider_name}] Płatność przyjęta!")
        return f"stripe_tx_{int(datetime.now().timestamp())}"


class FakeTestProcessor:
    @property
    def provider_name(self) -> str:
        return "Testowy"

    def process_payment(self, amount: float, currency: str) -> str:
        return "test_ok"


def wykonaj_platnosc(processor: PaymentProcessor, kwota: float):
    """Funkcja akceptuje dowolny obiekt pasujący do protokołu"""
    print(f"Procesor: {processor.provider_name}")
    tx = processor.process_payment(kwota, "PLN")
    print(f"Transakcja: {tx}\n")
```



```
stripe = StripeProcessor()
testowy = FakeTestProcessor()

wykonaj_platnosc(stripe, 149.99)
wykonaj_platnosc(testowy, 9.99)
```

Zalety Protocol:

- Nie musisz pisać class X(PaymentProcessor)
- Działa z istniejącymi klasami i bibliotekami trzecimi
- Najlepsze do dużych projektów i typowania statycznego (mypy, pyright)

Lekcja 8

Temat: Funkcje zaawansowane: Lambdy, map, filter, reduce; dekoratory

Funkcje lambda

Funkcje lambda **to anonimowe funkcje**, które można definiować w jednej linii. Przydatne do tworzenia prostych funkcji na potrzeby jednorazowego użycia, np. W **połączeniu z innymi funkcjami jak map, filter czy sorted. Nie mają nazwy i składają się z słowa kluczowego lambda, argumentów, dwukropka i wyrażenia.**

Przykład zastosowania: Sortowanie listy słowników po wartości klucza. Załóżmy, że mamy listę osób i chcemy posortować je po wieku.

```
osoby = [{'imie': 'Anna', 'wiek': 25}, {'imie': 'Jan', 'wiek': 30}, {'imie': 'Maria', 'wiek': 22}]
posortowane = sorted(osoby, key=lambda x: x['wiek'])
print(posortowane)
```

Przykład: Obliczenie kwadratów liczb w liście.

```
kwadraty = list(map(lambda x: x**2, [1, 2, 3, 4]))
print(kwadraty)
```

Funkcja map

Funkcja **map** stosuje podaną funkcję do każdego elementu iterowalnego (np. listy, tupli) i **zwraca iterator z wynikami**. Często łączy się ją z lambda dla zwięzłości.

Przykład zastosowania: Podwojenie elementów listy.

```
liczby = [1, 2, 3, 4]
podwojone = list(map(lambda x: x * 2, liczby))
print(podwojone)
```

Przykład: Konwersja temperatur z Celsjusza na Fahrenheita dla listy wartości.

Konwersja z **Celsjusza (°C)** na **Fahrenheita (°F)** działa według prostego wzoru matematycznego:

$$^{\circ}\text{F} = (^{\circ}\text{C} \times 9/5) + 32$$

Skale różnią się w dwóch rzeczach:

1. **Wielkość stopnia**
 - a. $100^{\circ}\text{C} = 180^{\circ}\text{F}$
 $\rightarrow 1^{\circ}\text{C} = 9/5^{\circ}\text{F}$ (czyli 1.8°F)
2. **Punkt zerowy**
 - a. $0^{\circ}\text{C} = 32^{\circ}\text{F}$
 $\rightarrow$ dlatego dodajemy **+32**

Przykłady

$$0^{\circ}\text{C} \rightarrow (0 \times 9/5) + 32 = 32^{\circ}\text{F}$$

$$25^{\circ}\text{C} \rightarrow (25 \times 9/5) + 32 = 77^{\circ}\text{F}$$

$$100^{\circ}\text{C} \rightarrow (100 \times 9/5) + 32 = 212^{\circ}\text{F}$$

```
celsjusze = [0, 10, 20, 30]
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsjusze))
print(fahrenheit)
```

Funkcja filter

Funkcja **filter** filtruje elementy iterowalnego na podstawie warunku podanego w funkcji (zwracającej True/False). Zwraca iterator z elementami spełniającymi warunek.

Przykład zastosowania: Wybór parzystych liczb z listy.

```
liczby = [1, 2, 3, 4, 5, 6]
parzyste = list(filter(lambda x: x % 2 == 0, liczby))
print(parzyste)
```

Przykład: Filtrowanie słów dłuższych niż 3 litery z listy.

```
slova = ['kot', 'pies', 'slon', 'ptak', 'ryba']
dlugie = list(filter(lambda s: len(s) > 3, slova))
print(dlugie)
```

Funkcja reduce

Funkcja reduce (z modułu functools) redukuje iterowalny do pojedynczej wartości, stosując funkcję kumulacyjną do elementów. Wymaga importu: `from functools import reduce`.

Ogólna składnia

`reduce(function, iterable)`

- **function** → funkcja z dwoma argumentami
- **iterable** → lista / tuple / inna kolekcja

Python bierze elementy **po kolei i łączy je w jeden wynik**.

Przykład zastosowania: Obliczenie sumy elementów listy.

```
from functools import reduce
liczby = [1, 2, 3, 4]
suma = reduce(lambda x, y: x + y, liczby)
print(suma)
```

Przykład: Znalezienie maksymalnej wartości w liście.

```
from functools import reduce
liczby = [1, 3, 2, 5, 4]
maks = reduce(lambda x, y: x if x > y else y, liczby)
print(maks)
```

Różnica między map, filter, reduce

Funkcja	Co robi
---------	---------

map	zmienia każdy element
filter	wybiera elementy
reduce	redukuje wszystko do jednej wartości

Dekoratory

Dekoratory **to funkcje, które modyfikują lub rozszerzają zachowanie innych funkcji bez zmiany ich kodu**. Definiuje się je z użyciem **@dekorator nad funkcją**. Są meta-funkcjami, które zwracają nową funkcję.

Idea dekoratora

Dekorator:

- bierze **funkcję jako argument**
- **opakowuje ją dodatkową logiką**
- zwraca **nową funkcję**

Czyli zamiast zmieniać funkcję, **dodajemy warstwę wokół niej**.

Przykład

```
def moj_dekorator(func):  
    def wrapper():  
        print("Coś przed funkcją")  
        func()  
        print("Coś po funkcji")  
    return wrapper
```

```
@moj_dekorator  
def powitanie():  
    print("Witaj!")
```

```
powitanie()
```

Przykład zastosowania: Dekorator mierzący czas wykonania funkcji.

```
import time  
  
def miernik_czasu(func):  
    def wrapper(*args, **kwargs):  
        start = time.time()  
        wynik = func(*args, **kwargs)  
        koniec = time.time()  
        print(f"Czas wykonania: {koniec - start} sekund")  
        return wynik  
    return wrapper  
  
@miernik_czasu  
def wolna_funkcja(n):  
    time.sleep(n)
```

```
        return "Gotowe"

print(wolna_funkcja(2))
```

Przykład: Dekorator dodający logowanie.

```
def loguj(func):
    def wrapper(*args, **kwargs):
        print(f"Wywołano {func.__name__} z argumentami {args}")
        return func(*args, **kwargs)
    return wrapper

@loguj
def dodaj(a, b):
    return a + b

print(dodaj(3, 5))
```

- Oddanie projektów